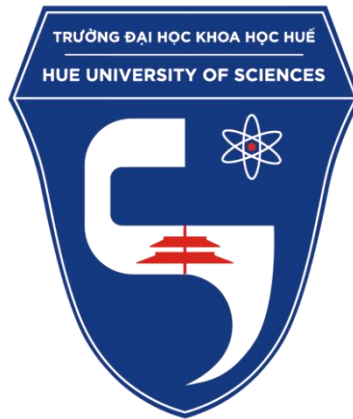


ĐẠI HỌC HUẾ
TRƯỜNG ĐẠI HỌC KHOA HỌC



BÀI TIỂU LUẬN

ĐỀ TÀI: HỆ THỐNG GIÁM SÁT ĐỘ RUNG VỚI ESP32

Huế, 2026

Danh mục từ viết tắt

ADC	Bộ chuyển đổi tương tự - số (Analog to Digital Converter)
DMP	Bộ xử lý chuyển động số (Digital Motion Processor)
ESP32	Dòng vi điều khiển có tích hợp Wi-Fi/Bluetooth của Espressif
GPIO	Chân vào/ra đa dụng (General Purpose Input Output)
I2C	Giao thức truyền thông nối tiếp liên mạch (Inter-Integrated Circuit)
IoT	Internet vạn vật (Internet of Things)
LCD	Màn hình tinh thể lỏng (Liquid Crystal Display)
MPU6050	Cảm biến 6 trục gồm gia tốc kế và con quay hồi chuyển
MQTT	Giao thức truyền tin nhẹ cho ứng dụng IoT (Message Queuing Telemetry Transport)

Mục lục

PHẦN MỞ ĐẦU	1
1. Lý do chọn đề tài.....	1
2. Mục tiêu nghiên cứu	1
3. Đối tượng và phạm vi nghiên cứu	2
4. Phương pháp nghiên cứu	2
5. Kết cấu tiểu luận	3
CHƯƠNG I: TỔNG QUAN	4
1.1. Giới thiệu IoT.....	4
1.2. Giám sát rung động	4
1.3. Ứng dụng thực tế	5
1.4. Đặt vấn đề.....	5
1.5. Kết luận chương.....	5
CHƯƠNG II: CƠ SỞ LÝ THUYẾT	6
2.1. ESP32	6
2.2. MPU6050	6
2.3. Cơ sở lý thuyết về rung động	7
2.4. Giao thức truyền thông I2C.....	8
2.5. Công cụ sử dụng	9
2.6. Kết luận chương.....	9
CHƯƠNG III: THIẾT KẾ HỆ THỐNG	10
3.1. Yêu cầu hệ thống	10
3.2. Sơ đồ khối hệ thống.....	10
3.3. Kết nối phần cứng	11
3.4. Quy trình xử lý dữ liệu.....	12
3.5. Chức năng giám sát từ xa	12
3.6. Giao diện	13
3.7. Đánh giá thiết kế hệ thống	14
3.8. Kết luận chương.....	15
CHƯƠNG IV: THỰC NGHIỆM	15
4.1. Môi trường thực nghiệm.....	15
4.2. Nội dung và quy trình thực nghiệm	16
4.2.1. Lắp ráp hệ thống	16

4.2.2. Nạp chương trình và kiểm tra kết nối	16
4.2.3. Tiến hành đo trong các trạng thái khác nhau	16
4.2.4. Tiêu chí lựa chọn ngưỡng cảnh báo	17
4.3. Kết quả.....	18
4.3.1. Trạng thái bình thường.....	18
4.3.2. Trạng thái rung nhẹ	18
4.3.3. Trạng thái rung vừa	18
4.3.4. Trạng thái rung mạnh.....	19
4.4. Đánh giá kết quả thực nghiệm	20
4.5. Hướng phát triển của hệ thống	21
4.6. Kết luận chương.....	22
PHẦN KẾT LUẬN VÀ KIẾN NGHỊ	22
5.1. Kết luận.....	22
5.2. Hướng phát triển	23

PHẦN MỞ ĐẦU

1. Lý do chọn đề tài

Trong thực tế, rung động là một thông số quan trọng phản ánh trạng thái hoạt động của máy móc, thiết bị và nhiều đối tượng cần được theo dõi. Việc phát hiện sớm các rung động bất thường có ý nghĩa lớn trong giám sát tình trạng thiết bị, cảnh báo nguy cơ hư hỏng, va chạm hoặc các tác động trái phép, từ đó góp phần nâng cao hiệu quả quản lý và bảo vệ tài sản.

Cùng với sự phát triển của Internet of Things (IoT), các hệ thống giám sát ngày càng được yêu cầu theo hướng nhỏ gọn, chi phí thấp, dễ triển khai và có khả năng kết nối không dây. ESP32 là vi điều khiển phù hợp cho các ứng dụng IoT nhờ được tích hợp Wi-Fi, khả năng xử lý tốt và dễ lập trình [1]. Trong khi đó, MPU6050 là cảm biến có khả năng đo gia tốc và vận tốc góc theo ba trục, đáp ứng tốt yêu cầu thu thập dữ liệu rung động và chuyển động [2].

Xuất phát từ những lý do trên, đề tài “Hệ thống giám sát độ rung với ESP32” được lựa chọn nhằm xây dựng một mô hình giám sát rung động có tính thực tiễn, dễ triển khai và có khả năng mở rộng trong các ứng dụng cảnh báo, bảo vệ thiết bị và giám sát từ xa.

2. Mục tiêu nghiên cứu

Đề tài hướng đến xây dựng một hệ thống giám sát độ rung sử dụng ESP32 và cảm biến MPU6050 nhằm phát hiện các rung động bất thường của đối tượng được theo dõi. Cụ thể, các mục tiêu nghiên cứu gồm:

- Tìm hiểu đặc điểm phần cứng và khả năng ứng dụng của vi điều khiển ESP32 trong hệ thống nhúng và IoT [1].
- Tìm hiểu nguyên lý hoạt động của cảm biến MPU6050 và khả năng đo gia tốc theo ba trục X, Y và Z [2].
- Thiết kế và lắp ráp mô hình phần cứng để thu thập dữ liệu gia tốc theo thời gian thực.
- Xây dựng thuật toán xử lý dữ liệu dựa trên độ lớn vector gia tốc tổng hợp, kết hợp hiệu chuẩn giá trị nền và lấy trung bình mẫu để giảm nhiễu.

- Thiết lập các ngưỡng cảnh báo nhằm phân loại trạng thái bình thường, rung nhẹ, rung vừa và rung mạnh.
- Đánh giá khả năng hoạt động của hệ thống thông qua thực nghiệm ở các trạng thái khác nhau, từ đó nhận xét ưu điểm, hạn chế và đề xuất hướng phát triển.

Ngoài ra, hệ thống còn được tích hợp chức năng gửi cảnh báo từ xa thông qua nền tảng Telegram. Khi phát hiện rung động vượt ngưỡng nguy hiểm, ESP32 sẽ tự động gửi thông báo đến người dùng thông qua kết nối Wi-Fi. Chức năng này giúp nâng cao tính ứng dụng thực tiễn của hệ thống trong các bài toán giám sát từ xa theo định hướng IoT.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là vi điều khiển ESP32, cảm biến gia tốc MPU6050 và bài toán giám sát rung động trong các hệ thống nhúng có khả năng mở rộng theo định hướng IoT.

Phạm vi nghiên cứu của đề tài tập trung vào việc xây dựng một mô hình giám sát rung động cơ bản có khả năng: thu thập dữ liệu gia tốc theo ba trục, tính toán cường độ rung động tổng hợp, so sánh với ngưỡng cảnh báo và hiển thị kết quả theo thời gian thực trên Serial Monitor, LCD hoặc thiết bị cảnh báo cục bộ. Trong phạm vi thực hiện của bài tiểu luận, hệ thống chưa triển khai đầy đủ chức năng truyền dữ liệu lên nền tảng giám sát trực tuyến; nội dung này được xem là hướng phát triển mở rộng trong tương lai. Ngoài ra, đề tài chưa đi sâu vào phân tích phổ rung, chẩn đoán hư hỏng công nghiệp chuyên sâu hoặc các thuật toán xử lý tín hiệu nâng cao.

Trong phạm vi đề tài, hệ thống tập trung xây dựng mô hình giám sát rung động cơ bản, bao gồm thu thập dữ liệu gia tốc ba trục, tính toán độ rung tổng hợp, so sánh với ngưỡng cảnh báo và hiển thị kết quả trên Serial Monitor, LCD hoặc thiết bị cảnh báo cục bộ.

Hệ thống chưa triển khai đầy đủ các nền tảng giám sát trực tuyến như MQTT hoặc Dashboard (Node-RED), tuy nhiên đã tích hợp chức năng cảnh báo từ xa qua Telegram. Đây là cơ sở để phát triển mở rộng trong tương lai.

4. Phương pháp nghiên cứu

Trong quá trình thực hiện đề tài, các phương pháp nghiên cứu sau được sử dụng:

- Phương pháp nghiên cứu tài liệu: thu thập, tổng hợp và phân tích các tài liệu liên quan đến ESP32, MPU6050, rung động và hệ thống IoT [1], [2], [4], [5].
- Phương pháp thiết kế hệ thống: xây dựng sơ đồ khối, lựa chọn linh kiện, thiết kế kết nối phần cứng và xây dựng luồng xử lý phần mềm.
- Phương pháp thực nghiệm: lắp ráp mô hình, nạp chương trình, đo thử trong các điều kiện khác nhau và đánh giá kết quả.
- Phương pháp so sánh, đánh giá: so sánh dữ liệu ở các trạng thái bình thường, rung nhẹ và rung mạnh để xác định hiệu quả hoạt động của hệ thống.

5. Kết cấu tiểu luận

Ngoài phần mở đầu và kết luận, nội dung bài tiểu luận gồm 4 chương chính.

Chương 1 trình bày tổng quan về IoT và giám sát rung động.

Chương 2 trình bày cơ sở lý thuyết liên quan đến ESP32, MPU6050, rung động và giao thức I2C.

Chương 3 trình bày thiết kế phần cứng và phần mềm của hệ thống.

Chương 4 trình bày quá trình thực nghiệm, kết quả đo và đánh giá.

CHƯƠNG I: TỔNG QUAN

1.1. Giới thiệu IoT

Internet of Things (IoT) là mạng lưới các thiết bị vật lý được tích hợp cảm biến, phần mềm và công nghệ kết nối, cho phép chúng thu thập, trao đổi và xử lý dữ liệu thông qua Internet mà không cần sự can thiệp trực tiếp của con người. IoT ngày càng được ứng dụng rộng rãi trong nhiều lĩnh vực như nhà thông minh, y tế, giao thông, nông nghiệp và công nghiệp.

Vai trò của IoT không chỉ dừng lại ở việc kết nối thiết bị mà còn giúp tự động hóa quy trình, nâng cao hiệu quả vận hành, tối ưu chi phí và hỗ trợ giám sát theo thời gian thực. Trong đề tài này, IoT đóng vai trò là nền tảng giúp hệ thống giám sát độ rung có thể mở rộng từ đo lường tại chỗ sang giám sát từ xa, hiển thị dữ liệu trên nền tảng trực tuyến và gửi cảnh báo khi phát hiện rung động bất thường.

1.2. Giám sát rung động

Rung động là sự dao động của một vật thể quanh vị trí cân bằng. Trong thực tế, rung động có thể xuất hiện trong quá trình hoạt động của máy móc, thiết bị hoặc khi có tác động cơ học từ bên ngoài. Việc giám sát rung động nhằm theo dõi trạng thái chuyển động của đối tượng, từ đó phát hiện các dấu hiệu bất thường có thể ảnh hưởng đến độ an toàn và độ ổn định của hệ thống.

Giám sát rung động có vai trò quan trọng trong việc đánh giá tình trạng hoạt động của thiết bị. Khi rung động vượt quá mức bình thường, đó có thể là dấu hiệu của va chạm, lỏng linh kiện, mất ổn định hoặc tác động trái phép. Nhờ đó, hệ thống giám sát rung động có thể hỗ trợ cảnh báo sớm, giúp người sử dụng có biện pháp xử lý kịp thời.

Trong phạm vi đề tài này, giám sát rung động được tiếp cận theo hướng đơn giản và thực tiễn, phù hợp với các ứng dụng quy mô nhỏ như theo dõi thiết bị, bảo vệ tài sản, giám sát cửa, xe máy hoặc các vật dụng có giá trị. Đây là hướng tiếp cận phù hợp với khả năng của ESP32 và cảm biến MPU6050, đồng thời đáp ứng yêu cầu xây dựng một hệ thống có chi phí thấp và dễ triển khai.

1.3. Ứng dụng thực tế

Hệ thống giám sát độ rung sử dụng ESP32 và MPU6050 có thể được ứng dụng trong nhiều tình huống thực tế khác nhau.

Trong công nghiệp quy mô nhỏ, hệ thống có thể được dùng để theo dõi rung động của máy móc nhằm phát hiện các trạng thái hoạt động bất thường. Trong đời sống, hệ thống có thể ứng dụng để bảo vệ xe máy, cửa, kết sắt hoặc các thiết bị có giá trị; khi phát hiện rung động hoặc dịch chuyển bất thường, hệ thống sẽ đưa ra cảnh báo. Ngoài ra, nhờ khả năng kết nối không dây của ESP32, hệ thống còn phù hợp với các mô hình nhà thông minh và giám sát tài sản từ xa.

Những ứng dụng trên cho thấy đề tài không chỉ có ý nghĩa về mặt nghiên cứu mà còn có tính thực tiễn cao, dễ mở rộng và phù hợp với xu hướng phát triển của các hệ thống giám sát thông minh hiện nay.

1.4. Đặt vấn đề

Trong thực tế, nhiều thiết bị và tài sản có thể chịu rung động bất thường do va chạm, tác động cơ học hoặc trạng thái làm việc không ổn định. Nếu các rung động này không được phát hiện kịp thời, thiết bị có thể bị suy giảm độ bền, giảm hiệu quả vận hành hoặc phát sinh rủi ro hư hỏng. Vì vậy, việc xây dựng một hệ thống có khả năng đo, xử lý và cảnh báo rung động là cần thiết.

Với ưu điểm chi phí thấp, dễ lập trình và tích hợp nhiều giao tiếp ngoại vi, ESP32 phù hợp để làm bộ xử lý trung tâm cho hệ thống giám sát. Kết hợp với cảm biến MPU6050 có khả năng đo gia tốc theo ba trục, hệ thống có thể thu thập dữ liệu chuyển động theo thời gian thực và xác định mức độ rung động của đối tượng được theo dõi.

Từ đó, đề tài tập trung giải quyết câu hỏi chính: liệu có thể xây dựng một mô hình giám sát rung động đơn giản, chi phí thấp, có khả năng phát hiện bất thường và đưa ra cảnh báo kịp thời bằng ESP32 và MPU6050 hay không. Trên cơ sở đó, hệ thống được định hướng mở rộng theo hướng kết nối và giám sát từ xa trong các ứng dụng IoT.

1.5. Kết luận chương

Chương 1 đã trình bày khái quát về Internet of Things (IoT), làm rõ vai trò của IoT trong việc kết nối, thu thập và xử lý dữ liệu từ các thiết bị thông minh. Bên cạnh đó, nội

dung chương cũng đã giới thiệu khái niệm giám sát rung động, ý nghĩa của việc theo dõi rung động trong phát hiện trạng thái bất thường và khả năng ứng dụng trong thực tế.

Ngoài ra, chương này còn phân tích tính ứng dụng của hệ thống giám sát độ rung sử dụng ESP32 và MPU6050 trong các lĩnh vực như giám sát thiết bị, bảo vệ tài sản và cảnh báo từ xa. Trên cơ sở đó, vấn đề nghiên cứu đã được đặt ra một cách rõ ràng, làm nền tảng cho các chương tiếp theo về cơ sở lý thuyết, thiết kế hệ thống và thực nghiệm.

CHƯƠNG II: CƠ SỞ LÝ THUYẾT

2.1. ESP32

ESP32 là dòng vi điều khiển tích hợp hệ thống trên chip (System on Chip - SoC) do hãng Espressif Systems phát triển, được sử dụng rộng rãi trong các ứng dụng nhúng và Internet of Things (IoT). Điểm nổi bật của ESP32 là được tích hợp sẵn kết nối Wi-Fi và Bluetooth, giúp thiết bị dễ dàng giao tiếp không dây với các hệ thống khác mà không cần bổ sung thêm module truyền thông riêng.

Về cấu hình phần cứng, ESP32 sở hữu bộ xử lý hai nhân (dual-core), tốc độ xung nhịp có thể lên đến 240 MHz, bộ nhớ SRAM lớn và hỗ trợ nhiều chuẩn giao tiếp ngoại vi như I2C, SPI, UART, ADC, PWM. Ngoài ra, ESP32 còn có khả năng tiết kiệm năng lượng tốt, phù hợp với các hệ thống giám sát hoạt động liên tục hoặc các thiết bị cần tối ưu điện năng tiêu thụ.

Trong đề tài này, ESP32 đóng vai trò là bộ xử lý trung tâm của hệ thống. Nhiệm vụ chính của vi điều khiển là nhận dữ liệu từ cảm biến MPU6050 thông qua giao tiếp I2C, thực hiện xử lý dữ liệu, tính toán mức độ rung động, so sánh với ngưỡng đã cài đặt và xuất dữ liệu ra thiết bị hiển thị hoặc hệ thống giám sát từ xa. Nhờ khả năng xử lý tương đối mạnh và tích hợp Wi-Fi, ESP32 là lựa chọn phù hợp để xây dựng hệ thống giám sát độ rung theo hướng nhỏ gọn, chi phí thấp và có khả năng mở rộng.

2.2. MPU6050

MPU6050 là cảm biến chuyển động 6 trục, tích hợp 3 trục gia tốc kế (Accelerometer) và 3 trục con quay hồi chuyển (Gyroscope) trong cùng một chip. Đây là cảm biến phổ biến trong các ứng dụng đo chuyển động, góc nghiêng, rung động và điều khiển cân bằng nhờ kích thước nhỏ gọn, giá thành thấp và khả năng giao tiếp đơn giản.

Gia tốc kế trong MPU6050 dùng để đo gia tốc tuyến tính theo ba trục X, Y, Z, trong khi con quay hồi chuyển dùng để đo vận tốc góc quanh ba trục tương ứng. Ngoài ra, MPU6050 còn tích hợp bộ xử lý chuyển động số DMP (Digital Motion Processor), hỗ trợ xử lý dữ liệu chuyển động trong một số ứng dụng nâng cao. Trong phạm vi đề tài này, hệ thống chủ yếu khai thác dữ liệu gia tốc để xác định mức độ rung động của đối tượng cần giám sát.

Về nguyên lý hoạt động, MPU6050 sử dụng công nghệ MEMS (Micro-Electro-Mechanical Systems). Khi cảm biến chịu tác động của gia tốc hoặc chuyển động quay, cấu trúc cơ điện tử bên trong sẽ thay đổi, dẫn đến sự thay đổi điện dung. Từ đó, cảm biến chuyển đổi đại lượng cơ học thành tín hiệu điện và xuất ra dữ liệu số tương ứng trên các trục đo.

MPU6050 hỗ trợ giao tiếp I2C, có thể hoạt động với điện áp phổ biến từ 3,3V đến 5V tùy module, dải đo gia tốc có thể lựa chọn như $\pm 2g$, $\pm 4g$, $\pm 8g$ và $\pm 16g$, với độ phân giải ADC 16-bit. Các đặc tính này cho phép cảm biến đáp ứng tốt yêu cầu đo rung động trong các mô hình giám sát quy mô nhỏ.

Trong đề tài, MPU6050 được gắn trực tiếp lên đối tượng cần theo dõi để thu thập dữ liệu gia tốc theo thời gian thực. Dữ liệu này sau đó được truyền đến ESP32 để xử lý và xác định trạng thái rung động của hệ thống.

2.3. Cơ sở lý thuyết về rung động

Rung động là sự dao động cơ học của một vật thể quanh vị trí cân bằng. Trong hệ thống giám sát độ rung, đại lượng được sử dụng để xác định rung động là gia tốc. Cảm biến MPU6050 cung cấp giá trị gia tốc tức thời theo ba trục X, Y, Z. Khi đối tượng đứng yên hoặc hoạt động bình thường, các giá trị này thường dao động trong một khoảng nhỏ. Khi có rung động mạnh hoặc tác động bất thường, biên độ gia tốc đo được sẽ thay đổi rõ rệt [2].

Để đánh giá cường độ rung động một cách tổng quát và không phụ thuộc vào hướng tác động, có thể sử dụng độ lớn của vector gia tốc tổng hợp. Công thức tính như sau:

$$A = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

Trong đó:

- a_x : gia tốc theo trục X
- a_y : gia tốc theo trục Y
- a_z : gia tốc theo trục Z
- A : độ lớn vector gia tốc tổng hợp

Sau khi hiệu chuẩn giá trị nền khi cảm biến ở trạng thái đứng yên, độ rung của hệ thống được xác định theo công thức:

$$D=|A-A_0|$$

Trong đó:

- D : giá trị độ rung
- A : gia tốc tổng hợp tức thời
- A_0 : giá trị gia tốc nền sau hiệu chuẩn

Giá trị D càng lớn thì mức độ rung động của đối tượng càng mạnh. Dựa trên giá trị này, hệ thống có thể thiết lập các ngưỡng để phân biệt trạng thái bình thường và trạng thái bất thường.

2.4. Giao thức truyền thông I2C

I2C (Inter-Integrated Circuit) là giao thức truyền thông nối tiếp đồng bộ, thường được sử dụng để kết nối vi điều khiển với các cảm biến, bộ nhớ và nhiều thiết bị ngoại vi khác. Một ưu điểm nổi bật của I2C là chỉ cần hai dây tín hiệu để truyền dữ liệu, bao gồm dây SDA (Serial Data) dùng cho dữ liệu và dây SCL (Serial Clock) dùng cho xung nhịp.

Trong giao tiếp I2C, thiết bị điều khiển trung tâm đóng vai trò Master, còn các thiết bị ngoại vi đóng vai trò Slave. Master sẽ tạo xung nhịp, gửi địa chỉ thiết bị cần giao tiếp và thực hiện việc đọc hoặc ghi dữ liệu. Mỗi thiết bị Slave trên bus I2C có một địa chỉ riêng để phân biệt với các thiết bị khác.

Trong hệ thống của đề tài, ESP32 đóng vai trò là thiết bị Master, còn MPU6050 là thiết bị Slave. ESP32 sẽ gửi lệnh đến MPU6050 để khởi tạo cảm biến, sau đó đọc dữ liệu gia tốc từ các thanh ghi của cảm biến theo chu kỳ nhất định. Việc sử dụng giao thức

I2C giúp giảm số lượng dây kết nối, đơn giản hóa phần cứng và thuận tiện cho quá trình lập trình.

2.5. Công cụ sử dụng

Để triển khai đề tài, một số công cụ phần mềm được sử dụng nhằm hỗ trợ lập trình, nạp chương trình và giám sát dữ liệu.

Arduino IDE là môi trường phát triển tích hợp phổ biến dành cho các hệ thống nhúng. Phần mềm này hỗ trợ viết mã nguồn bằng ngôn ngữ C/C++, biên dịch chương trình và nạp trực tiếp xuống vi điều khiển ESP32. Ngoài ra, Arduino IDE còn cung cấp Serial Monitor và Serial Plotter để theo dõi dữ liệu cảm biến dưới dạng số hoặc đồ thị theo thời gian thực.

Node-RED là công cụ lập trình trực quan dựa trên luồng (flow-based programming), thường được sử dụng trong các ứng dụng IoT để kết nối, xử lý dữ liệu và xây dựng giao diện Dashboard. Nếu được tích hợp trong hệ thống, Node-RED sẽ hỗ trợ hiển thị dữ liệu rung động một cách trực quan hơn, giúp người dùng theo dõi từ xa thông qua mạng Wi-Fi.

Ngoài hai công cụ trên, trong quá trình lập trình hệ thống có thể sử dụng thêm các thư viện như Wire.h hoặc Adafruit_MPU6050 để hỗ trợ giao tiếp với cảm biến MPU6050, giảm độ phức tạp của việc truy cập các thanh ghi phần cứng.

2.6. Kết luận chương

Chương 2 đã trình bày những cơ sở lý thuyết quan trọng phục vụ cho việc xây dựng hệ thống giám sát độ rung. Cụ thể, chương đã giới thiệu về vi điều khiển ESP32, cảm biến MPU6050, khái niệm rung động, công thức xác định cường độ rung dựa trên dữ liệu gia tốc và giao thức truyền thông I2C giữa các thiết bị.

Bên cạnh đó, chương cũng đã nêu ra các công cụ phần mềm hỗ trợ cho quá trình lập trình, thu thập và hiển thị dữ liệu. Đây là nền tảng lý thuyết cần thiết để tiến hành thiết kế phần cứng, xây dựng thuật toán xử lý và triển khai hệ thống trong chương tiếp theo.

CHƯƠNG III: THIẾT KẾ HỆ THỐNG

3.1. Yêu cầu hệ thống

Để đáp ứng mục tiêu của đề tài, hệ thống giám sát độ rung cần bảo đảm một số yêu cầu cơ bản về phần cứng và phần mềm.

Trước hết, hệ thống phải có khả năng thu thập dữ liệu gia tốc theo ba trục X, Y, Z từ cảm biến MPU6050 một cách liên tục và tương đối chính xác theo thời gian thực. Dữ liệu thu được phải phản ánh được mức độ rung động của đối tượng cần giám sát.

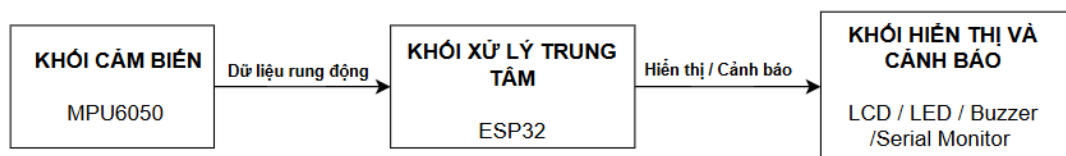
Tiếp theo, hệ thống cần xử lý dữ liệu để xác định cường độ rung động tổng quát. Việc xử lý bao gồm đọc dữ liệu cảm biến, quy đổi giá trị đo, tính toán độ lớn vector gia tốc tổng hợp và thực hiện lọc nhiễu cơ bản để dữ liệu ổn định hơn.

Ngoài ra, hệ thống cần có khả năng hiển thị dữ liệu lên máy tính hoặc giao diện giám sát, đồng thời đưa ra cảnh báo khi giá trị rung động vượt quá ngưỡng cho phép. Cảnh báo có thể được thể hiện bằng thông báo trên Serial Monitor, LCD, LED hoặc buzzer.

Cuối cùng, hệ thống cần có cấu trúc đơn giản, dễ lắp đặt, dễ lập trình, chi phí thấp và có khả năng mở rộng theo hướng kết nối IoT trong tương lai.

3.2. Sơ đồ khối hệ thống

Hệ thống giám sát độ rung được xây dựng gồm ba khối chức năng chính: khối cảm biến, khối xử lý trung tâm và khối hiển thị/cảnh báo.



Hình 3.1. Sơ đồ khối hệ thống giám sát độ rung sử dụng ESP32 và MPU6050

- Khối cảm biến: sử dụng MPU6050 để thu thập dữ liệu gia tốc theo ba trục X, Y, và Z.
- Khối xử lý trung tâm: sử dụng ESP32 để nhận dữ liệu từ cảm biến, xử lý và xác định mức độ rung động.

- Khởi hiển thị và cảnh báo: hiển thị dữ liệu trên Serial Monitor, Serial Plotter hoặc LCD; đồng thời phát cảnh báo khi rung động vượt ngưỡng cài đặt.

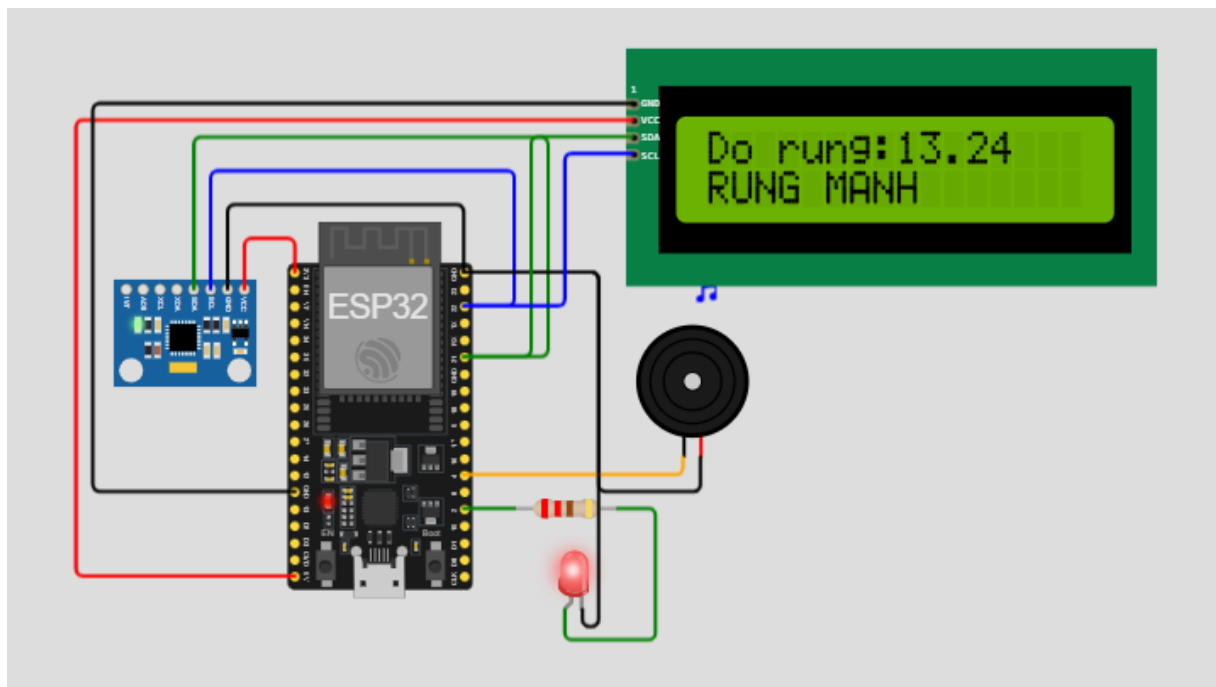
Trong đó, MPU6050 đóng vai trò thu thập tín hiệu rung động từ môi trường, ESP32 thực hiện xử lý và truyền kết quả đến thiết bị hiển thị hoặc cảnh báo. Sơ đồ khối này cho thấy hệ thống có cấu trúc đơn giản nhưng vẫn đáp ứng được các chức năng cơ bản của một hệ thống giám sát độ rung.

3.3. Kết nối phần cứng

MPU6050 sử dụng giao tiếp I2C để kết nối với ESP32. Các chân kết nối cơ bản như sau:

- VCC của MPU6050 nối với chân 3.3V của ESP32
- GND của MPU6050 nối với GND của ESP32
- SDA của MPU6050 nối với GPIO 21 của ESP32
- SCL của MPU6050 nối với GPIO 22 của ESP32

Cách kết nối này đảm bảo ESP32 có thể giao tiếp ổn định với cảm biến MPU6050 thông qua I2C.



Hình 3.2. Sơ đồ kết nối phần cứng giữa ESP32, MPU6050, LCD, LED và buzzer trên nền tảng Wokwi

3.4. Quy trình xử lý dữ liệu

Dữ liệu được lấy mẫu theo chu kỳ thời gian thực từ 10–20 ms tùy cấu hình hệ thống. Phần mềm của hệ thống được xây dựng với các bước xử lý cơ bản như sau:

Bước 1: Khởi tạo hệ thống, bao gồm giao tiếp Serial, giao tiếp I2C, cảm biến MPU6050 và các thiết bị cảnh báo.

Bước 2: Đọc dữ liệu gia tốc theo ba trục a_x , a_y , a_z từ cảm biến MPU6050.

Bước 3: Tính độ lớn vector gia tốc tổng hợp:

$$A = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

Bước 4: Hiệu chuẩn giá trị nền A_0 khi cảm biến ở trạng thái đứng yên.

Bước 5: Tính độ rung theo công thức:

$$D = |A - A_0|$$

Bước 6: Lấy trung bình nhiều mẫu liên tiếp để giảm nhiễu.

Bước 7: So sánh giá trị D với các ngưỡng đã thiết lập để xác định trạng thái bình thường, rung nhẹ, rung vừa hoặc rung mạnh.

Bước 8: Hiển thị kết quả lên Serial Monitor/LCD và kích hoạt LED hoặc buzzer khi phát hiện rung vượt ngưỡng.

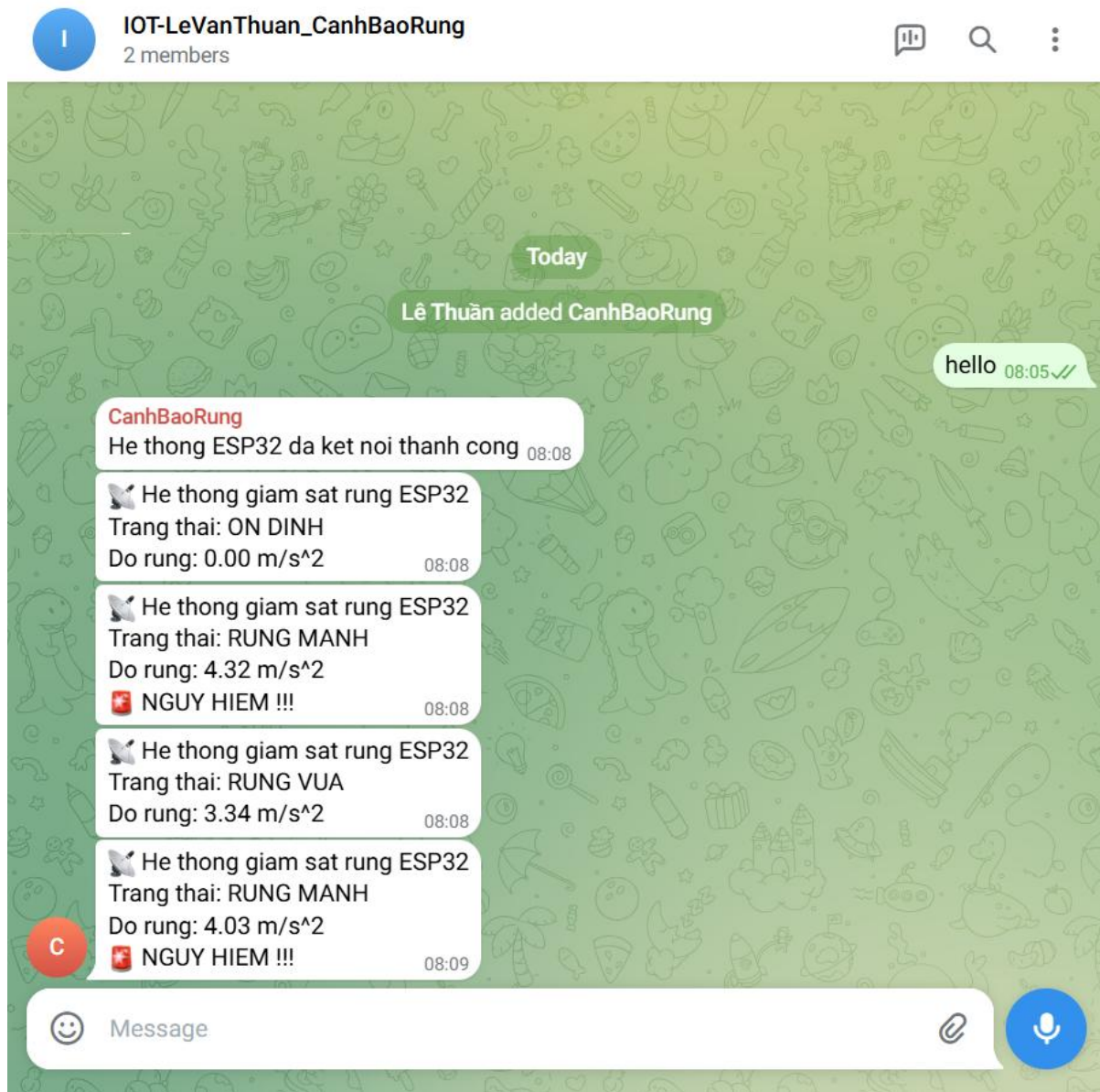
Bước 9: Lặp lại quá trình đo theo chu kỳ thời gian thực.

3.5. Chức năng giám sát từ xa

Bên cạnh các chức năng cảnh báo tại chỗ bằng LED và buzzer, hệ thống còn được tích hợp khả năng gửi cảnh báo từ xa thông qua ứng dụng Telegram. Khi giá trị độ rung vượt ngưỡng cao, ESP32 sẽ gửi yêu cầu HTTP đến API của Telegram Bot để truyền thông báo đến người dùng.

Nội dung thông báo bao gồm giá trị độ rung đo được và trạng thái cảnh báo tương ứng. Nhờ đó, người dùng có thể theo dõi tình trạng hệ thống từ xa thông qua thiết bị di động có kết nối Internet.

Chức năng này giúp nâng cao khả năng giám sát của hệ thống, đồng thời thể hiện định hướng ứng dụng theo mô hình IoT, khi dữ liệu không chỉ được xử lý cục bộ mà còn được truyền tải qua mạng để phục vụ theo dõi và cảnh báo từ xa.



Hình 3.3. Hệ thống cảnh báo rung động ứng dụng ESP32, MPU6050, LCD, LED và buzzer trên Telegram

3.6. Giao diện

Để người dùng dễ dàng theo dõi trạng thái của hệ thống, dữ liệu đo được có thể hiển thị theo nhiều hình thức khác nhau.

Trước hết, Serial Monitor trong Arduino IDE cho phép hiển thị các giá trị gia tốc theo từng trục và giá trị rung động tổng hợp dưới dạng số. Đây là cách đơn giản nhất để kiểm tra hoạt động của hệ thống trong quá trình thử nghiệm.

Bên cạnh đó, Serial Plotter có thể được sử dụng để biểu diễn dữ liệu dưới dạng đồ thị theo thời gian thực. Nhờ đó, người dùng có thể quan sát trực quan sự thay đổi của tín hiệu rung động và dễ dàng nhận biết khi xuất hiện dao động bất thường.

Nếu hệ thống được mở rộng theo hướng IoT, dữ liệu có thể được gửi lên Node-RED Dashboard hoặc giao diện web để theo dõi từ xa. Khi đó, có thể sử dụng các thành phần như biểu đồ đường (Chart), đồng hồ đo (Gauge) hoặc hộp cảnh báo để nâng cao khả năng giám sát.

Về chức năng cảnh báo, trong phiên bản cơ bản, hệ thống có thể hiển thị cảnh báo bằng văn bản trên Serial Monitor khi giá trị rung vượt ngưỡng. Trong phiên bản mở rộng, có thể bổ sung còi báo, đèn LED hoặc gửi thông báo qua mạng để tăng tính thực tiễn.

3.7. Đánh giá thiết kế hệ thống

Thiết kế hệ thống giám sát độ rung sử dụng ESP32 và MPU6050 có nhiều ưu điểm như cấu trúc đơn giản, dễ lắp ráp, chi phí thấp và phù hợp với các mô hình thử nghiệm hoặc ứng dụng nhỏ. Hệ thống có khả năng đo rung động theo thời gian thực, xử lý dữ liệu nhanh và có thể mở rộng sang các ứng dụng giám sát từ xa nhờ kết nối Wi-Fi của ESP32.

Tuy nhiên, do sử dụng cảm biến phổ thông và thuật toán xử lý cơ bản, hệ thống vẫn còn một số hạn chế nhất định. Độ chính xác của dữ liệu có thể bị ảnh hưởng bởi nhiều môi trường, sai số cảm biến hoặc cách gắn cảm biến lên đối tượng đo. Ngoài ra, hệ thống hiện mới dừng ở mức xác định cường độ rung tổng quát, chưa thực hiện phân tích sâu như nhận dạng tần số rung hoặc phân loại nguyên nhân gây rung.

Mặc dù vậy, với mục tiêu của đề tài, thiết kế này vẫn đáp ứng tốt yêu cầu xây dựng một mô hình giám sát độ rung đơn giản, hiệu quả và có tính ứng dụng thực tế.

3.8. Kết luận chương

Chương 3 đã trình bày quá trình thiết kế hệ thống giám sát độ rung sử dụng ESP32 và MPU6050, bao gồm yêu cầu hệ thống, sơ đồ khối, thiết kế phần cứng, kết nối giữa các linh kiện, thuật toán xử lý dữ liệu và phương thức hiển thị, cảnh báo.

Thông qua việc xác định rõ chức năng của từng khối và quy trình hoạt động của hệ thống, chương này đã tạo cơ sở để triển khai lắp ráp mô hình, lập trình điều khiển và tiến hành thực nghiệm trong chương tiếp theo.

CHƯƠNG IV: THỰC NGHIỆM

4.1. Môi trường thực nghiệm

Để kiểm tra khả năng hoạt động của hệ thống giám sát độ rung, quá trình thực nghiệm được tiến hành trên mô hình gồm vi điều khiển ESP32 và cảm biến MPU6050. Hệ thống được triển khai trong cả môi trường mô phỏng và môi trường thực tế nhằm đánh giá độ ổn định cũng như khả năng phản hồi của hệ thống.

Về thiết bị, mô hình sử dụng một máy tính xách tay có cài đặt phần mềm Arduino IDE hoặc Visual Studio Code kết hợp PlatformIO để biên dịch và nạp chương trình cho ESP32. Phần cứng gồm board ESP32 DevKit V1, cảm biến MPU6050, dây nối jumper, breadboard và cáp USB dùng để cấp nguồn cũng như truyền dữ liệu. Ngoài ra, để thuận tiện trong quá trình kiểm tra nguyên lý hoạt động ban đầu, hệ thống có thể được mô phỏng trên nền tảng Wokwi trước khi triển khai trên mạch thật.

Về điều kiện thực nghiệm, hệ thống được khảo sát trong hai trạng thái chính. Trạng thái thứ nhất là môi trường tĩnh, trong đó cảm biến được giữ cố định để xác định giá trị nền ban đầu và quan sát độ ổn định của tín hiệu. Trạng thái thứ hai là môi trường rung mô phỏng, trong đó cảm biến được tạo dao động bằng thao tác lắc tay hoặc đặt trên một bề mặt có rung động nhẹ để kiểm tra khả năng phát hiện rung và phản ứng cảnh báo của hệ thống.

Trong mỗi trạng thái khảo sát, hệ thống được thu thập dữ liệu liên tục trong một khoảng thời gian đủ để quan sát sự ổn định của tín hiệu và xác định giá trị trung bình. Việc giữ cố định vị trí lắp cảm biến trong suốt quá trình thử nghiệm giúp hạn chế sai số phát sinh do thay đổi tư thế đo.

Việc lựa chọn cả mô hình mô phỏng và mô hình thử nghiệm thực tế giúp tăng tính tin cậy của kết quả, đồng thời hỗ trợ người thực hiện dễ dàng kiểm tra, hiệu chỉnh và mở rộng hệ thống trong các giai đoạn tiếp theo.

4.2. Nội dung và quy trình thực nghiệm

4.2.1. Lắp ráp hệ thống

Trước khi tiến hành đo đạc, các linh kiện được kết nối theo sơ đồ phần cứng đã thiết kế ở Chương 3. Cảm biến MPU6050 được nối với ESP32 thông qua giao tiếp I2C, trong đó chân SDA của cảm biến được nối với chân GPIO 21 của ESP32, chân SCL nối với GPIO 22, chân VCC nối với nguồn 3.3V và chân GND nối với mass chung của hệ thống. Sau khi hoàn tất phần cứng, hệ thống được kiểm tra sơ bộ để bảo đảm các dây nối đúng vị trí, không bị lỏng hoặc chập mạch, nhằm tránh sai số trong quá trình thu thập dữ liệu.

4.2.2. Nạp chương trình và kiểm tra kết nối

Chương trình điều khiển được viết bằng ngôn ngữ C/C++ trên Arduino IDE hoặc Visual Studio Code. Sau khi hoàn thành mã nguồn, chương trình được biên dịch và nạp xuống vi điều khiển ESP32 thông qua cáp USB.

Sau bước nạp chương trình, hệ thống được mở cổng Serial Monitor để kiểm tra kết nối giữa ESP32 và MPU6050. Nếu cảm biến hoạt động bình thường, dữ liệu gia tốc theo ba trục X, Y, Z sẽ được hiển thị liên tục trên màn hình. Đồng thời, Serial Plotter cũng được sử dụng để quan sát dạng đồ thị của tín hiệu rung động theo thời gian thực.

Việc kiểm tra này giúp xác nhận rằng hệ thống đã khởi tạo thành công giao tiếp I2C, đọc được dữ liệu từ cảm biến và sẵn sàng cho quá trình đánh giá.

4.2.3. Tiến hành đo trong các trạng thái khác nhau

Để đánh giá khả năng hoạt động của hệ thống, quá trình thực nghiệm được thực hiện trong **bốn trạng thái: bình thường, rung nhẹ, rung vừa và rung mạnh**. Ở mỗi trạng thái, cảm biến được đặt trong cùng một điều kiện lắp đặt và hệ thống tiến hành thu thập dữ liệu liên tục trong một khoảng thời gian nhất định.

Trong quá trình đo, hệ thống thu thập dữ liệu gia tốc theo ba trục a_x , a_y , a_z được đọc từ MPU6050, sau đó tính giá trị gia tốc tổng hợp theo công thức:

$$A = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

Từ giá trị này, hệ thống xác định độ rung bằng cách so sánh với giá trị nền sau hiệu chuẩn theo biểu thức:

$$D = |A - A_0|$$

Đối với mỗi trạng thái, đề tài ghi nhận giá trị trung bình, giá trị lớn nhất và trạng thái cảnh báo tương ứng. Việc so sánh các nhóm số liệu này cho phép đánh giá khả năng phân biệt giữa trạng thái ổn định và trạng thái rung bất thường của hệ thống.

Kết quả ở mỗi trạng thái được tổng hợp từ nhiều lần quan sát liên tiếp, trong đó giá trị độ rung trung bình phản ánh xu hướng chung của tín hiệu, còn giá trị lớn nhất thể hiện mức dao động cực đại xuất hiện trong quá trình thử nghiệm.

4.2.4. Tiêu chí lựa chọn ngưỡng cảnh báo

Các ngưỡng cảnh báo của hệ thống được xác định dựa trên quá trình đo thử trong nhiều trạng thái hoạt động khác nhau của cảm biến. Trước tiên, cảm biến được giữ cố định để hệ thống thực hiện hiệu chuẩn và xác định giá trị nền ban đầu. Sau đó, hệ thống được khảo sát trong các trạng thái rung nhẹ, rung vừa và rung mạnh bằng cách tạo dao động với biên độ tăng dần.

Trong quá trình thử nghiệm, các giá trị độ rung được quan sát trên Serial Monitor và so sánh giữa các trạng thái để lựa chọn các mức ngưỡng phù hợp. Tiêu chí đặt ra là: trạng thái bình thường không được gây cảnh báo giả; trạng thái rung nhẹ chỉ phản ánh sự dao động nhỏ; trong khi trạng thái rung vừa và rung mạnh phải được phát hiện rõ ràng để kích hoạt cảnh báo tương ứng.

Trên cơ sở đó, hệ thống sử dụng ba mức ngưỡng: 0,8 m/s² cho rung nhẹ, 2,0 m/s² cho rung vừa và 4,0 m/s² cho rung mạnh. Các giá trị này phù hợp với mô hình thử nghiệm hiện tại và giúp cân bằng giữa khả năng phát hiện rung động bất thường với việc hạn chế cảnh báo sai trong điều kiện bình thường.

4.3. Kết quả

4.3.1. Trạng thái bình thường

Khi cảm biến được giữ yên trên bề mặt cố định, các giá trị gia tốc đo được trên ba trục có dao động rất nhỏ. Giá trị tổng hợp A thường xấp xỉ 1g do ảnh hưởng của gia tốc trọng trường. Trên Serial Plotter, tín hiệu biểu diễn tương đối ổn định, đường đồ thị gần như nằm ngang và không xuất hiện các đỉnh dao động lớn.

Kết quả này cho thấy hệ thống có khả năng đọc dữ liệu ổn định trong điều kiện không có rung động đáng kể, đồng thời phản ánh đúng đặc trưng của cảm biến gia tốc trong trạng thái đứng yên.

4.3.2. Trạng thái rung nhẹ

Khi cảm biến chịu tác động rung nhẹ, các giá trị gia tốc trên các trục bắt đầu thay đổi quanh giá trị nền ban đầu. Biên độ dao động của tín hiệu chưa lớn nhưng đã có sự biến thiên rõ rệt so với trạng thái bình thường. Trên đồ thị, đường tín hiệu xuất hiện các gợn sóng nhỏ liên tiếp, thể hiện sự thay đổi liên tục của gia tốc theo thời gian.

Ở trạng thái này, hệ thống vẫn theo dõi tốt sự thay đổi của dữ liệu. Nếu ngưỡng cảnh báo được đặt ở mức phù hợp, hệ thống có thể phân biệt được rung nhẹ và rung mạnh, tránh phát cảnh báo giả khi chỉ có dao động nhỏ từ môi trường.

4.3.3. Trạng thái rung vừa

Khi cảm biến chịu tác động rung ở mức trung bình, các giá trị gia tốc trên ba trục thay đổi rõ rệt hơn so với trạng thái rung nhẹ nhưng chưa đạt đến mức dao động mạnh và liên tục như trạng thái rung mạnh. Giá trị gia tốc tổng hợp lúc này tăng lên đáng kể, thường xuyên vượt qua ngưỡng rung nhẹ và tiến gần hoặc vượt ngưỡng rung vừa đã được thiết lập trong hệ thống.

Trên Serial Plotter, tín hiệu ở trạng thái rung vừa xuất hiện các đỉnh dao động có biên độ lớn hơn so với trạng thái rung nhẹ, đồng thời tần suất biến thiên cũng cao hơn. Tuy nhiên, dạng tín hiệu vẫn chưa dày đặc và đột biến mạnh như ở trạng thái rung mạnh. Điều này cho thấy hệ thống có khả năng phân biệt tương đối rõ giữa rung nhẹ, rung vừa và rung mạnh dựa trên giá trị độ rung tổng hợp.

Ở trạng thái này, hệ thống bắt đầu kích hoạt mức cảnh báo trung gian, cụ thể là bật đèn LED nhưng chưa kích hoạt buzzer. Cách cảnh báo này phù hợp với mục tiêu phân loại rung động theo nhiều mức, giúp người dùng nhận biết được trạng thái bất thường nhưng chưa đến mức nghiêm trọng nhất. Kết quả đo trong bảng thực nghiệm cho thấy trạng thái rung vừa có giá trị độ rung trung bình khoảng $2,63 \text{ m/s}^2$ và giá trị lớn nhất đạt khoảng $3,42 \text{ m/s}^2$, phù hợp với ngưỡng cảnh báo đã thiết lập cho hệ thống.

4.3.4. Trạng thái rung mạnh

Khi cảm biến bị rung lắc mạnh, dữ liệu gia tốc thay đổi nhanh và liên tục trên cả ba trục. Giá trị tổng hợp $A_{tăng}$ lên rõ rệt, nhiều thời điểm vượt qua ngưỡng cài đặt của hệ thống. Trên Serial Plotter, đồ thị xuất hiện các đỉnh dao động lớn, biên độ cao và tần suất thay đổi nhanh hơn so với hai trạng thái trước.

Trong trường hợp này, hệ thống đã phát hiện được trạng thái bất thường và đưa ra cảnh báo theo đúng thiết kế. Điều này cho thấy mô hình có khả năng phản hồi theo thời gian thực và đáp ứng được yêu cầu phát hiện rung động lớn.

4.4. Đánh giá kết quả thực nghiệm

Trạng thái	Giá trị độ rung trung bình (m/s^2)	Giá trị lớn nhất (m/s^2)	LED	Buzzer	Nhận xét
Bình thường	0.12	0.28	Tắt	Tắt	Tín hiệu ổn định, không cảnh báo
Rung nhẹ	1.08	1.64	Tắt	Tắt	Có dao động rõ nhưng chưa vượt ngưỡng rung vừa
Rung vừa	2.63	3.42	Sáng	Tắt	Hệ thống phát hiện rung bất thường mức vừa
Rung mạnh	4.87	5.91	Sáng	Kêu	Hệ thống cảnh báo rõ rệt và liên tục

Bảng 4.1. Kết quả đo độ rung của hệ thống trong các trạng thái thực nghiệm

Từ Bảng 4.1 có thể thấy giá trị độ rung tăng dần theo từng mức tác động từ bình thường đến rung mạnh. Đồng thời, cơ chế cảnh báo của hệ thống cũng thay đổi tương ứng từ không cảnh báo, đến cảnh báo bằng LED, và cuối cùng là cảnh báo bằng cả LED và buzzer. Điều này cho thấy hệ thống không chỉ phát hiện được rung động mà còn có khả năng phân loại mức độ rung một cách tương đối rõ ràng.

Kết quả thực nghiệm cho thấy hệ thống có khả năng thu thập dữ liệu gia tốc theo thời gian thực và phản ánh được sự khác biệt giữa bốn trạng thái: bình thường, rung nhẹ, rung vừa và rung mạnh. Ở trạng thái bình thường, tín hiệu gia tốc dao động nhỏ quanh giá trị nền và không gây ra cảnh báo giả. Khi xuất hiện rung nhẹ, dữ liệu bắt đầu biến thiên rõ hơn nhưng vẫn nằm dưới mức cảnh báo trung gian. Đến trạng thái rung vừa, hệ thống đã có thể nhận biết mức dao động bất thường và kích hoạt cảnh báo bằng đèn

LED. Khi rung mạnh xảy ra, giá trị độ rung tăng rõ rệt, vượt ngưỡng cao nhất và hệ thống kích hoạt đồng thời LED và buzzer theo đúng thiết kế.

Về mặt kỹ thuật, mô hình cho thấy khả năng hoạt động ổn định với cấu trúc phần cứng đơn giản, chi phí thấp và dễ triển khai. Việc sử dụng ESP32 kết hợp MPU6050 đáp ứng tốt yêu cầu xây dựng một hệ thống giám sát rung động cơ bản trong các ứng dụng học tập, thử nghiệm và cảnh báo tại chỗ. Ngoài ra, thuật toán xử lý dựa trên gia tốc tổng hợp, hiệu chuẩn giá trị nền và lấy trung bình nhiều mẫu liên tiếp đã góp phần làm giảm nhiễu và tăng độ ổn định cho kết quả đo.

Một điểm nổi bật của hệ thống là khả năng kết hợp giữa cảnh báo cục bộ và cảnh báo từ xa thông qua LED và buzzer, hệ thống còn gửi thông báo qua Telegram khi phát hiện rung mạnh. Trong quá trình thực nghiệm, thông báo được gửi gần như tức thời với độ trễ nhỏ hơn 2 giây, chứng tỏ hệ thống có khả năng phản hồi nhanh và phù hợp với các ứng dụng giám sát từ xa.

Tuy nhiên, kết quả thực nghiệm cũng cho thấy hệ thống vẫn còn một số hạn chế. Độ chính xác của phép đo phụ thuộc vào cách gắn cảm biến, điều kiện môi trường và mức độ ổn định của đối tượng cần theo dõi. Mặt khác, phương pháp xử lý hiện tại chủ yếu dựa trên ngưỡng cài đặt thủ công nên mới phù hợp với mục tiêu phát hiện rung động bất thường ở mức cơ bản, chưa đủ để phân tích chuyên sâu về tần số rung, nguyên nhân gây rung hoặc chẩn đoán hư hỏng cơ khí.

Nhìn chung, trong phạm vi của bài tiểu luận, hệ thống đã đáp ứng được mục tiêu đề ra là xây dựng mô hình giám sát độ rung đơn giản, có khả năng phân loại nhiều mức rung động và đưa ra cảnh báo tương ứng theo thời gian thực. Đây là cơ sở phù hợp để phát triển các hướng mở rộng như giám sát từ xa, lưu trữ dữ liệu, gửi cảnh báo trực tuyến hoặc áp dụng các phương pháp xử lý tín hiệu nâng cao trong tương lai.

4.5. Hướng phát triển của hệ thống

Dựa trên kết quả thực nghiệm, hệ thống có thể được phát triển thêm theo nhiều hướng nhằm nâng cao tính ứng dụng trong thực tế.

Thứ nhất, có thể xây dựng giao diện giám sát trực tuyến bằng cách kết hợp ESP32 với giao thức MQTT và Node-RED Dashboard hoặc nền tảng web. Khi đó, dữ liệu rung

động sẽ không chỉ được hiển thị trên máy tính cục bộ mà còn có thể theo dõi từ xa thông qua mạng Internet.

Thứ hai, hệ thống có thể tích hợp chức năng gửi cảnh báo qua Telegram, email hoặc ứng dụng điện thoại. Khi phát hiện giá trị rung vượt ngưỡng, thiết bị sẽ tự động gửi thông báo đến người dùng, giúp tăng hiệu quả giám sát và phản ứng kịp thời.

Thứ ba, có thể bổ sung các thuật toán lọc tín hiệu như trung bình trượt, Complementary Filter hoặc Kalman Filter để giảm nhiễu, nâng cao độ chính xác và độ ổn định của dữ liệu đo. Đây là hướng cải tiến cần thiết nếu hệ thống được sử dụng trong môi trường có nhiều rung động phức tạp.

Ngoài ra, hệ thống cũng có thể được mở rộng thêm chức năng lưu trữ dữ liệu lên thẻ nhớ SD hoặc nền tảng đám mây để phục vụ cho quá trình thống kê, phân tích và đánh giá lịch sử rung động trong thời gian dài.

4.6. Kết luận chương

Chương 4 đã trình bày quá trình thực nghiệm và đánh giá hệ thống giám sát độ rung sử dụng ESP32 và MPU6050. Kết quả thử nghiệm cho thấy hệ thống có khả năng thu thập dữ liệu gia tốc theo thời gian thực, xử lý được các trạng thái rung động khác nhau và phát hiện được rung động bất thường khi vượt ngưỡng cài đặt.

Bên cạnh đó, chương cũng đã phân tích những ưu điểm, hạn chế và các hướng phát triển tiềm năng của hệ thống. Những kết quả này là cơ sở để khẳng định tính khả thi của đề tài, đồng thời làm tiền đề cho việc ứng dụng và mở rộng hệ thống trong tương lai.

PHẦN KẾT LUẬN VÀ KIẾN NGHỊ

5.1. Kết luận

Đề tài “Hệ thống giám sát độ rung sử dụng ESP32 và cảm biến MPU6050” đã xây dựng được một mô hình giám sát rung động cơ bản có khả năng thu thập dữ liệu gia tốc theo ba trục, xử lý dữ liệu theo thời gian thực và đưa ra cảnh báo khi phát hiện rung động vượt ngưỡng cài đặt. Kết quả thực nghiệm cho thấy ESP32 và MPU6050 là hai thành phần phù hợp để xây dựng các hệ thống giám sát rung động có chi phí thấp, dễ triển khai và mang tính ứng dụng thực tiễn [1], [2].

Thông qua quá trình thiết kế, lập trình và thực nghiệm, đề tài đã làm rõ nguyên lý đo rung dựa trên dữ liệu gia tốc, đồng thời chứng minh rằng mô hình có thể phân biệt được các trạng thái bình thường, rung nhẹ, rung vừa và rung mạnh trong phạm vi thử nghiệm. Bên cạnh đó, hệ thống có khả năng mở rộng theo hướng hiển thị trực quan, cảnh báo từ xa và tích hợp các nền tảng IoT trong các nghiên cứu hoặc ứng dụng tiếp theo.

Tuy nhiên, đề tài hiện vẫn dừng ở mức mô hình giám sát rung động cơ bản. Hệ thống chưa thực hiện phân tích rung chuyên sâu theo miền tần số, chưa tối ưu ngưỡng dựa trên tập dữ liệu thực nghiệm lớn và chưa triển khai đầy đủ hệ thống giám sát từ xa qua Internet. Những hạn chế này là cơ sở để phát triển và hoàn thiện hệ thống trong các nghiên cứu tiếp theo.

5.2. Hướng phát triển

Trong tương lai, hệ thống có thể được tiếp tục nghiên cứu và nâng cấp theo một số hướng sau:

- Nâng cao độ chính xác của dữ liệu bằng cách áp dụng các thuật toán lọc nhiễu như Kalman Filter, Complementary Filter hoặc các phương pháp xử lý tín hiệu số khác.
- Mở rộng khả năng giám sát từ xa bằng cách tích hợp giao thức MQTT, xây dựng Dashboard trên web hoặc ứng dụng điện thoại.
- Bổ sung chức năng gửi cảnh báo qua Telegram, email hoặc tin nhắn khi hệ thống phát hiện rung động bất thường.
- Tích hợp lưu trữ dữ liệu lên thẻ nhớ SD hoặc nền tảng đám mây để phục vụ việc thống kê và phân tích trong thời gian dài.
- Phát triển hệ thống theo hướng tối ưu phần cứng, giảm kích thước, giảm tiêu thụ năng lượng và tăng độ ổn định để có thể triển khai thực tế ở quy mô lớn hơn.

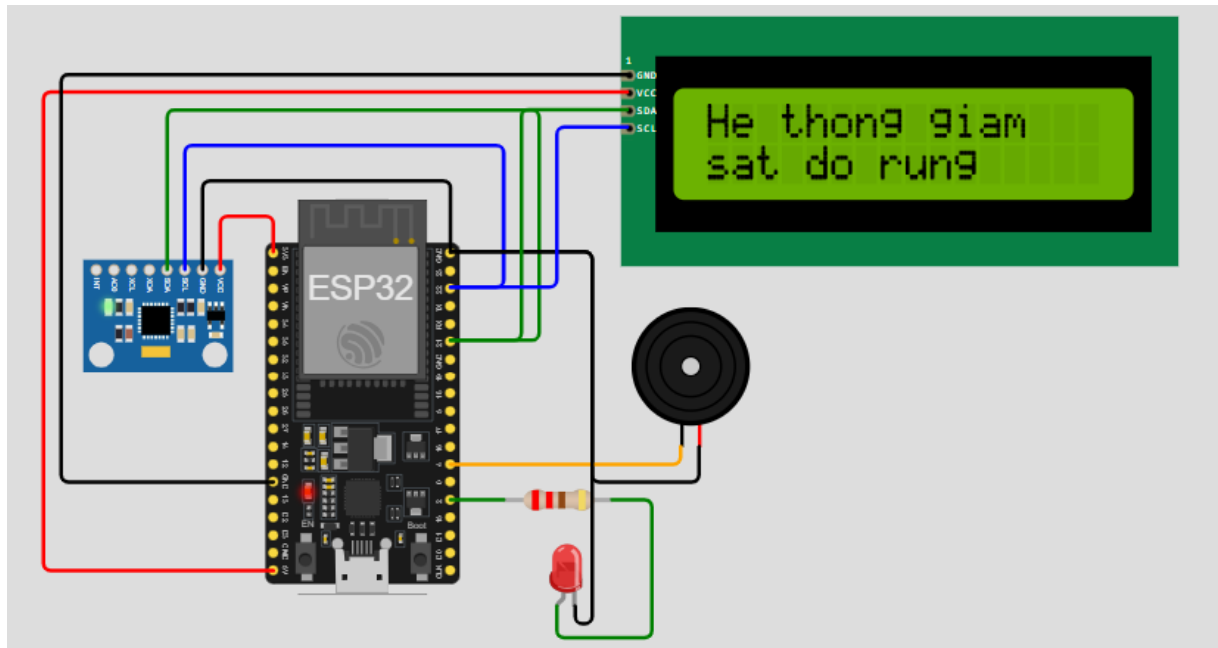
Với những hướng phát triển này, hệ thống giám sát độ rung sử dụng ESP32 và MPU6050 có thể trở thành nền tảng cho nhiều ứng dụng thông minh trong đời sống và kỹ thuật.

TÀI LIỆU THAM KHẢO

- [1] Espressif Systems, *ESP32-WROOM-32 Datasheet*, 2023. Espressif Systems. Truy cập ngày: 09/04/2026.
- [2] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification, Revision 3.4*, 2013. InvenSense Inc. Truy cập ngày: 09/04/2026.
- [3] Adafruit Industries, *Adafruit MPU6050 Library Documentation*, 2023. Truy cập ngày: 09/04/2026.
- [4] Arduino, *Arduino IDE Documentation*, 2024. <https://docs.arduino.cc/> (truy cập: 09/04/2026).
- [5] Node-RED, *Node-RED Documentation*, 2024. <https://nodered.org/docs/> (truy cập: 09/04/2026).

PHỤ LỤC

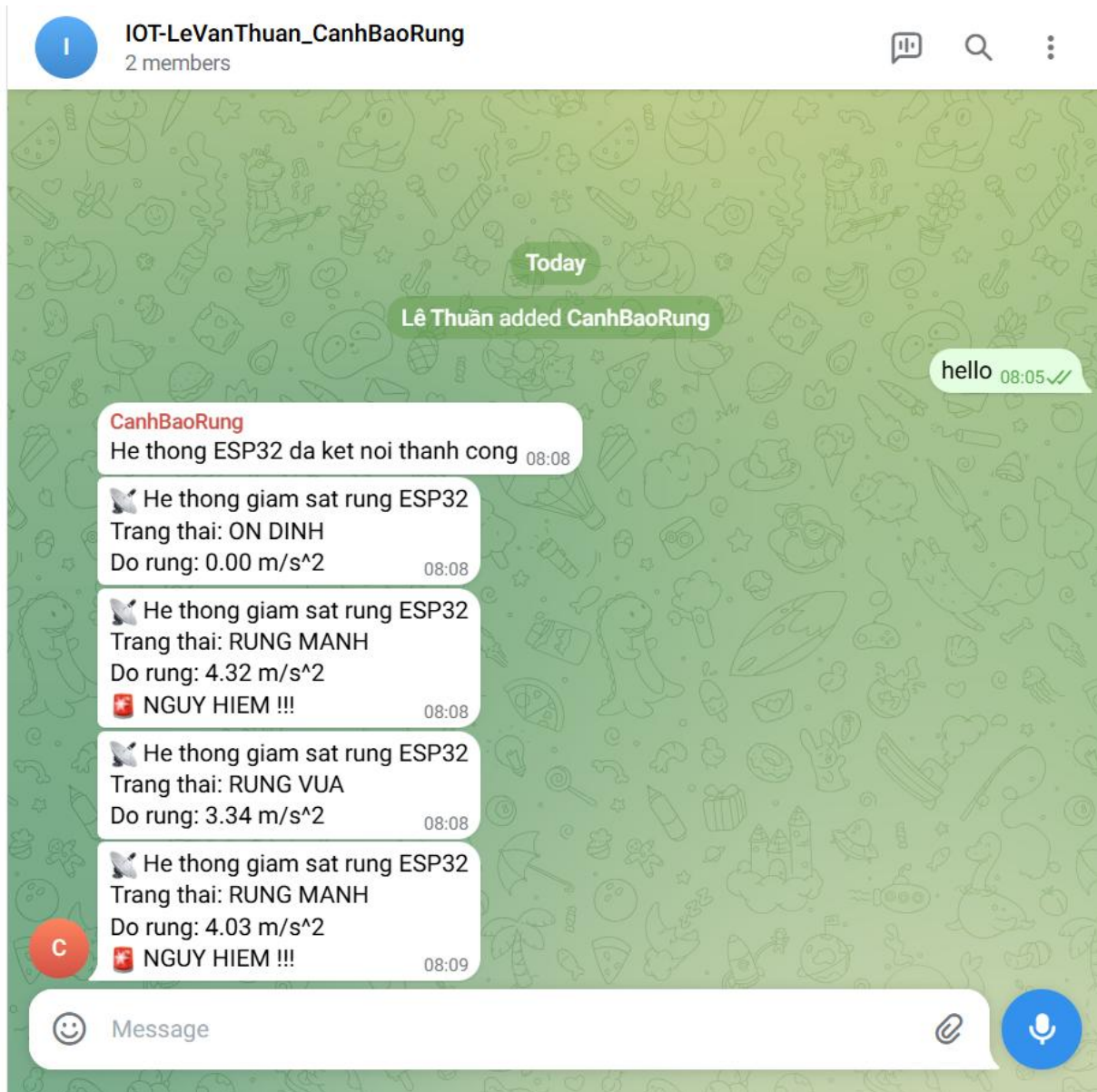
Phụ lục A: Hình ảnh mô hình và kết quả thực nghiệm



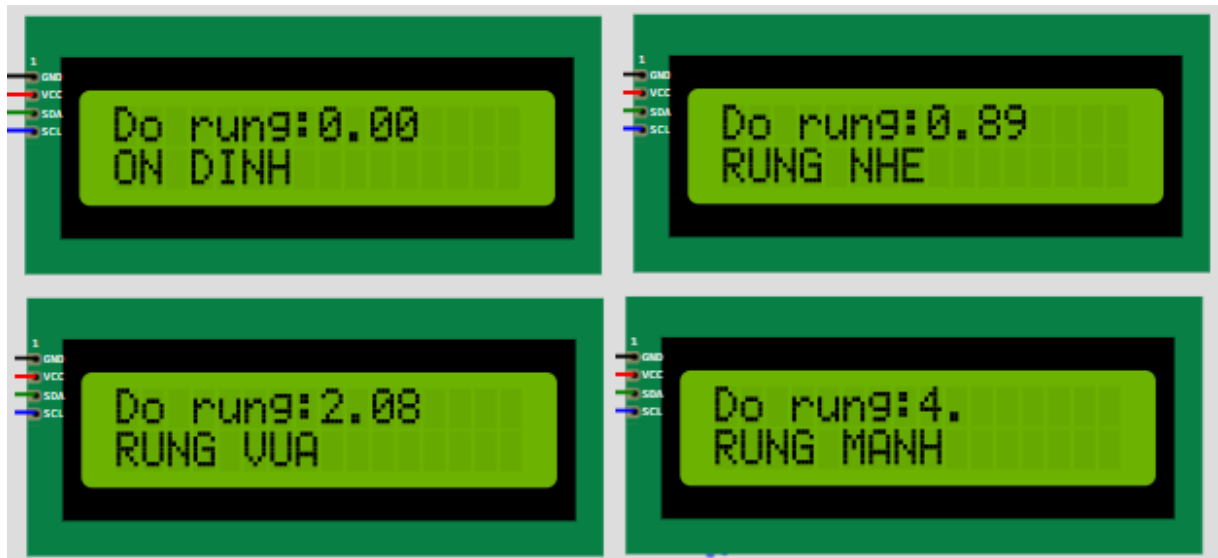
Hình A.1. Mô hình kết nối ESP32, MPU6050, LCD, LED và buzzer trên Wokwi

```
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Dang hieu chuan, giu cam bien dung yen...
Baseline = 9.807
He thong giam sat do rung bat dau...
[CANH BAO] Do rung: 5.892 m/s^2 | RUNG MANH
[CANH BAO] Do rung: 5.892 m/s^2 | RUNG MANH
[CANH BAO] Do rung: 5.892 m/s^2 | RUNG MANH
[CANH BAO] Do rung: 1.889 m/s^2 | RUNG NHE
[CANH BAO] Do rung: 1.889 m/s^2 | RUNG NHE
[CANH BAO] Do rung: 1.889 m/s^2 | RUNG NHE
[CANH BAO] Do rung: 1.889 m/s^2 | RUNG NHE
[CANH BAO] Do rung: 5.456 m/s^2 | RUNG MANH
[CANH BAO] Do rung: 2.529 m/s^2 | RUNG VUA
[CANH BAO] Do rung: 2.529 m/s^2 | RUNG VUA
[CANH BAO] Do rung: 2.529 m/s^2 | RUNG VUA
[CANH BAO] Do rung: 2.529 m/s^2 | RUNG VUA
```

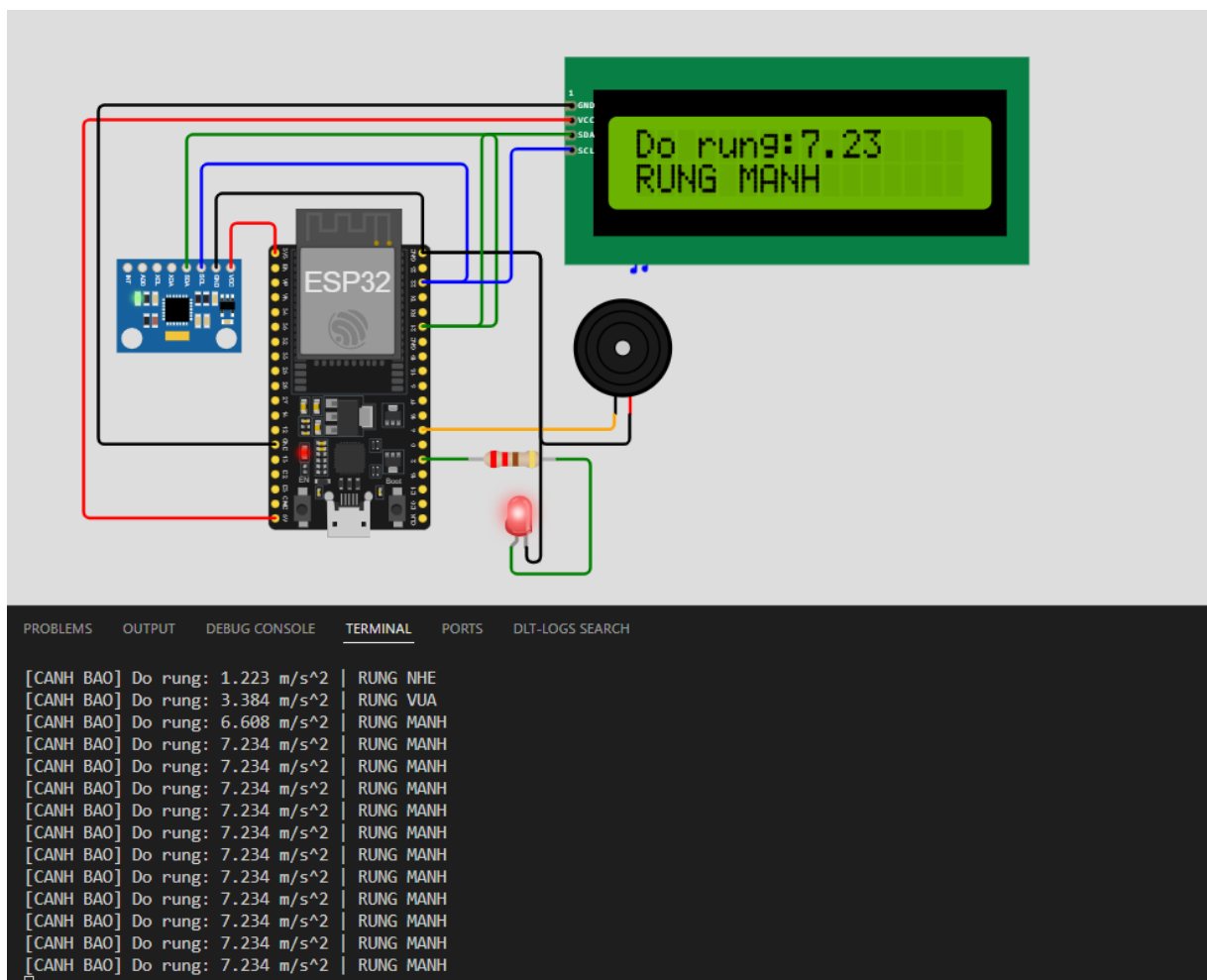
Hình A.2. Giao diện Serial Monitor trong quá trình hiệu chuẩn và đo độ rung



Hình A.3. Hệ thống cảnh báo rung động ứng dụng ESP32, MPU6050, LCD, LED và buzzer trên Telegram (minh họa lại Hình 3.3)



Hình A.4. Kết quả hiển thị trạng thái rung trên LCD



Hình A.5. Kết quả mô phỏng hệ thống trong môi trường Wokwi

Phụ lục B: Mã nguồn chương trình điều khiển ESP32 và MPU6050

```
#include <Arduino.h>

#include <Wire.h>

#include <math.h>

#include <Adafruit_MPU6050.h>

#include <Adafruit_Sensor.h>

#include <LiquidCrystal_I2C.h>


// WIFI + TELEGRAM

#include <WiFi.h>

#include <HTTPClient.h>


// ===== WIFI =====

const char* ssid = "Wokwi-GUEST";

const char* password = "";


// ===== TELEGRAM =====

String botToken = "8791087879:AAEIkG9nYjvSM0__4QdFcHjs5VWLur_A9M4";

String chatID = "-5252683483";


// ===== HARDWARE =====

Adafruit_MPU6050 mpu;

LiquidCrystal_I2C lcd(0x27, 16, 2);


const int LED_PIN = 2;

const int BUZZER_PIN = 4;

const int BUZZER_CHANNEL = 0;


// ===== THRESHOLD =====
```



```
float baseline = 0.0;
```

```
const float NGUONG_YEU = 0.8;
```

```
const float NGUONG_VUA = 2.0;
```

```
const float NGUONG_MANH = 4.0;
```

```
const int SO_MAU = 20;
```

```
// ===== CONTROL =====
```

```
String trangThaiCu = ""; // lưu trạng thái trước
```

```
unsigned long lanCapNhatLCD = 0;
```

```
// ===== ENCODE =====
```

```
String urlEncode(String msg) {
```

```
    String encoded = "";
```

```
    char c;
```

```
    char code0;
```

```
    char code1;
```

```
    for (int i = 0; i < msg.length(); i++) {
```

```
        c = msg.charAt(i);
```

```
        if (isalnum(c)) {
```

```
            encoded += c;
```

```
        } else {
```

```
            encoded += '%';
```

```
            code0 = (c >> 4) & 0xF;
```

```
            code1 = c & 0xF;
```

```
            encoded += (code0 > 9) ? char(code0 + 'A' - 10) : char(code0 + '0');
```

```

        encoded += (code1 > 9) ? char(code1 + 'A' - 10) : char(code1 + '0');
    }
}

return encoded;
}

// ===== TELEGRAM =====

void guiTelegram(String message) {
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;

        String url = "https://api.telegram.org/bot" + botToken +
            "/sendMessage?chat_id=" + chatID +
            "&text=" + urlEncode(message);

        http.begin(url);
        int httpCode = http.GET();

        Serial.print("HTTP Code: ");
        Serial.println(httpCode);

        if (httpCode > 0) {
            Serial.println("Gui Telegram thanh cong");
        } else {
            Serial.println("Loi gui Telegram");
        }

        http.end();
    }
}

```

```
}
```

```
// ===== MPU =====
```

```
float tinhDoRung() {
```

```
    sensors_event_t a, g, temp;
```

```
    mpu.getEvent(&a, &g, &temp);
```

```
    float tong = sqrt(
```

```
        a.acceleration.x * a.acceleration.x +
```

```
        a.acceleration.y * a.acceleration.y +
```

```
        a.acceleration.z * a.acceleration.z
```

```
    );
```

```
    return fabs(tong - baseline);
```

```
}
```

```
float locTrungBinh() {
```

```
    float tong = 0;
```

```
    for (int i = 0; i < SO_MAU; i++) {
```

```
        tong += tinhDoRung();
```

```
        delay(5);
```

```
    }
```

```
    return tong / SO_MAU;
```

```
}
```

```
// ===== CALIB =====
```

```
void hieuChuanCamBien() {
```

```
    lcd.clear();
```

```
    lcd.print("Hieu chuan...");
```

```
float tong = 0;
```

```
for (int i = 0; i < 100; i++) {  
    sensors_event_t a, g, temp;  
    mpu.getEvent(&a, &g, &temp);
```

```
float doLon = sqrt(  
    a.acceleration.x * a.acceleration.x +  
    a.acceleration.y * a.acceleration.y +  
    a.acceleration.z * a.acceleration.z  
);
```

```
tong += doLon;  
delay(20);  
}
```

```
baseline = tong / 100.0;
```

```
lcd.clear();  
lcd.print("Baseline:");  
lcd.setCursor(0, 1);  
lcd.print(baseline, 2);
```

```
delay(1500);  
}
```

```
// ===== ALERT =====
```

```
void tatCanhBao() {
```

```
digitalWrite(LED_PIN, LOW);  
ledcWriteTone(BUZZER_CHANNEL, 0);  
}
```

```
void canhBaoVua() {  
    digitalWrite(LED_PIN, HIGH);  
    ledcWriteTone(BUZZER_CHANNEL, 0);  
}
```

```
void canhBaoManh() {  
    digitalWrite(LED_PIN, HIGH);  
    ledcWriteTone(BUZZER_CHANNEL, 2000);  
}
```

```
// ===== LCD =====
```

```
void hienThiLCD(float doRung, String muc) {  
    lcd.setCursor(0, 0);  
    lcd.print("Do rung:    ");  
    lcd.setCursor(8, 0);  
    lcd.print(doRung, 2);  
  
    lcd.setCursor(0, 1);  
    lcd.print("    ");  
    lcd.setCursor(0, 1);  
    lcd.print(muc);  
}
```

```
// ===== SETUP =====
```

```
void setup() {
```

```
Serial.begin(115200);

Wire.begin(21, 22);


pinMode(LED_PIN, OUTPUT);


lcd.init();
lcd.backlight();


ledcSetup(BUZZER_CHANNEL, 2000, 8);
ledcAttachPin(BUZZER_PIN, BUZZER_CHANNEL);


// WIFI
WiFi.begin(ssid, password);
Serial.print("Dang ket noi WiFi");


while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}


Serial.println("\n WiFi OK");


guiTelegram("He thong ESP32 da ket noi thanh cong");


// MPU
if (!mpu.begin()) {
    lcd.print("MPU ERROR!");
    while (1);
}
```

```

mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
mpu.setGyroRange(MPU6050_RANGE_500_DEG);
mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

delay(1000);
hieuChuanCamBien();
}

// ===== LOOP =====

void loop() {
    float doRung = locTrungBinh();
    String muc = "ON DINH";

    if(doRung >= NGUONG_MANH) {
        muc = "RUNG MANH";
        canhBaoManh();
    }
    else if(doRung >= NGUONG_VUA) {
        muc = "RUNG VUA";
        canhBaoVua();
    }
    else if(doRung >= NGUONG_YEU) {
        muc = "RUNG NHE";
        tatCanhBao();
    }
    else {
        muc = "ON DINH";
        tatCanhBao();
    }
}

```

```
}
```

```
// CHỈ GỬI KHI TRẠNG THÁI THAY ĐỔI
```

```
if (muc != trangThaiCu) {
```

```
    String message = " ⚡ He thống giám sát rung ESP32\n";
```

```
    message += "Trang thai: " + muc + "\n";
```

```
    message += "Do rung: " + String(doRung, 2) + " m/s^2";
```

```
    if (muc == "RUNG MANH") {
```

```
        message += "\n NGUY HIEM !!!";
```

```
    }
```

```
    guiTelegram(message);
```

```
    trangThaiCu = muc;
```

```
}
```

```
// LCD
```

```
if (millis() - lanCapNhatLCD > 500) {
```

```
    hienThiLCD(doRung, muc);
```

```
    lanCapNhatLCD = millis();
```

```
}
```

```
delay(100);
```

```
}
```

Phụ lục C: Sơ đồ mô phỏng mạch hệ thống trên Wokwi (diagram.json)

```
{
```

```
  "version": 1,
```

```
  "author": "Thuần Lê",
```

```
  "editor": "wokwi",
```



```
"parts": [  
  { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 9.6, "left": -62.36, "attrs":  
    {} },  
  { "type": "wokwi-mpu6050", "id": "imu1", "top": 42.22, "left": -160.88, "attrs": {}  
    },  
  {  
    "type": "wokwi-led",  
    "id": "led1",  
    "top": 188.4,  
    "left": 81,  
    "attrs": { "color": "red", "flip": "1" }  
  },  
  {  
    "type": "wokwi-resistor",  
    "id": "r1",  
    "top": 167.15,  
    "left": 67.2,  
    "attrs": { "value": "220" }  
  },  
  { "type": "wokwi-buzzer", "id": "buzz1", "top": 60, "left": 136.2, "attrs": {} },  
  {  
    "type": "wokwi-lcd1602",  
    "id": "lcd1",  
    "top": -89.6,  
    "left": 130.4,  
    "attrs": { "pins": "i2c" }  
  }  
],  
"connections": [
```

```
[ "esp:TX", "$serialMonitor:RX", "", [] ],
[ "esp:RX", "$serialMonitor:TX", "", [] ],
[ "esp:3V3", "imu1:VCC", "red", [ "v-19.2", "h-38.33" ] ],
[ "esp:GND.2", "imu1:GND", "black", [ "v-38.4", "h-144.08" ] ],
[ "imu1:SDA", "esp:21", "green", [ "v-86.4", "h201.68", "v124.8" ] ],
[ "imu1:SCL", "esp:22", "blue", [ "v-67.2", "h172.88", "v76.8" ] ],
[ "esp:2", "r1:1", "green", [] ],
[ "r1:2", "led1:A", "green", [ "h21.6", "v74.75", "h-48" ] ],
[ "led1:C", "esp:GND.2", "black", [ "v9.6", "h9.2", "v-201.6" ] ],
[ "esp:4", "buzz1:1", "orange", [] ],
[ "buzz1:2", "esp:GND.2", "black", [ "v19.2", "h-58", "v-124.8" ] ],
[ "esp:5V", "lcd1:VCC", "red", [ "h-124.65", "v-259.3" ] ],
[ "esp:GND.1", "lcd1:GND", "black", [ "h-115.05", "v-220.8" ] ],
[ "esp:21", "lcd1:SDA", "green", [ "h38.4", "v-105.8" ] ],
[ "esp:22", "lcd1:SCL", "blue", [ "h57.6", "v-67.5" ] ]
],
"dependencies": {}
}
```

Phụ lục D: Cấu hình môi trường và thư viện dự án (platformio.ini)

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200

lib_deps =
  adafruit/Adafruit MPU6050
  adafruit/Adafruit Unified Sensor
  marcoschwartz/LiquidCrystal_I2C
```

PHIẾU ĐÁNH GIÁ TIỂU LUẬN**Học kỳ Năm học ...-...****SỐ PHÁCH:.....**

Cán bộ chấm thi 1	Cán bộ chấm thi 2
Nhận xét:	Nhận xét:
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
Điểm đánh giá của CBChT1:	Điểm đánh giá của CBChT2:
Bảng số:	Bảng số:
Bảng chữ:	Bảng chữ:

Điểm kết luận: Bảng số..... Bảng chữ:.....*Thừa Thiên Huế, ngày tháng năm 20...***CBChT1***(Ký và ghi rõ họ tên)***CBChT2***(Ký và ghi rõ họ tên)*